PROJECT REPORT: AFTERIMAGE

Nov 14, 2026 Submitted by Souryam Bandyopadhyay

AFTERIMAGE is a local-first search engine, written in C++20 with a Python/JavaScript web front end, that finds your own documents from half-remembered fragments such as a typo, a word stem, a vague sentence or "that PDF from last week".

Field	Entry
Project title	AFTERIMAGE: Human-Memory Document Retrieval System
Submitted by	Souryam Bandyopadhyay
Guide / Course	[Guide name, course: DSA / ADSA]
Languages	C++20 (engine), Python 3 (web adapter), JavaScript/HTML/CSS (UI)
Size of code	about 2,900 lines of C++ source, 740 lines of headers, 1,175 lines of tests, 1,530 lines of Python, 2,500 lines of web UI

Abstract

People rarely remember the exact name or exact words of a file they saved months ago. They remember a topic, a diagram, a few words, or roughly when they opened it. Ordinary file search (name matching) and plain keyword search both fail in that situation.

AFTERIMAGE solves this with an eight-module pipeline: it scans folders, extracts text from TXT, Markdown, PDF and images (using Poppler and Tesseract OCR), stores the text in SQLite, builds a positional inverted index in memory, retrieves candidates through five routes (exact, prefix, infix, typo-tolerant and phrase), ranks them with BM25 plus explainable boosts, and finally shows related documents using a concept graph. A second layer, AFTERIMAGE 2.0, adds a natural-language query parser, time-expression understanding ("last week"), page-level highlighted passages, an optional local-AI semantic search through Ollama embeddings, and a browser interface with a developer console.

All 12 automated test groups pass. Everything runs on the user's own machine; no document ever leaves it.

1. Introduction

1.1 Problem statement

A modern student or professional keeps thousands of files: notes, PDFs, scanned pages and screenshots. Finding one of them needs the exact filename or the exact keyword. Three things go wrong in practice:

- **Imperfect recall.** The user types "schedulng" or only "ntersection" and gets nothing.
- **Locked text.** Scanned PDFs and photos contain words that no keyword search can see until OCR is applied.
- **No context.** Results are a flat list. Nothing tells the user why a file matched, which page to open, or what other files are related.

1.2 Motivation

The name AFTERIMAGE comes from the faint image that remains after you look away from something. The system is designed around that idea: it retrieves a document from the trace the user still remembers, not from a perfect query.

1.3 Comparison with existing tools

Feature	OS file-name search	Plain keyword (grep-style)	Cloud AI search	AFTERIMAGE
Searches inside files	Partly	Yes (text only)	Yes	Yes (text, PDF, OCR)
Typo and fragment tolerance	No	No	Yes	Yes (fuzzy, prefix, infix)
Explains why a file matched	No	No	Rarely	Yes (per-route and per-score breakdown)
Understands "last week"	Limited	No	Yes	Yes
Related-document discovery	No	No	Yes	Yes (concept graph)
Works offline, private	Yes	Yes	No	Yes

2. Objectives, scope and technology

2.1 Objectives

1. Build a persistent library of documents with stable IDs that survive restarts.
2. Extract searchable text from TXT, Markdown, text-layer PDFs, scanned PDFs and images.
3. Implement the core data structures by hand: a positional inverted index, an ordered vocabulary for prefix search, and a document-concept graph.
4. Retrieve documents from imperfect fragments: stems, middle-of-word pieces, one-letter typos and exact phrases.
5. Rank results with a standard, explainable model (BM25) and show the score breakdown.
6. Suggest related documents the user did not ask for, using shared rare concepts.
7. Understand natural queries, time expressions and file-type words, and point to the exact page and passage.
8. Verify every module with automated unit, integration and command-line tests.

2.2 Scope

In scope: local single-user search over a folder tree, on macOS and Linux, through a terminal CLI and a browser UI on 127.0.0.1. Out of scope: multi-user access, cloud sync, handwriting recognition, and understanding diagrams or charts beyond the text OCR can read.

2.3 Requirements

Type	Requirement
Hardware	Any laptop or desktop; no GPU. About 2 GB RAM is enough for a library of a few thousand documents.
Operating system	macOS (primary, Spotlight app) or Linux. The process runner uses POSIX calls (posix_spawn, pipe, waitpid).
Build tools	C++20 compiler (Clang or GCC), CMake 3.20 or newer, SQLite3 development library
Document tools	Poppler (pdftotext, pdftoppm, pdfinfo) and Tesseract OCR

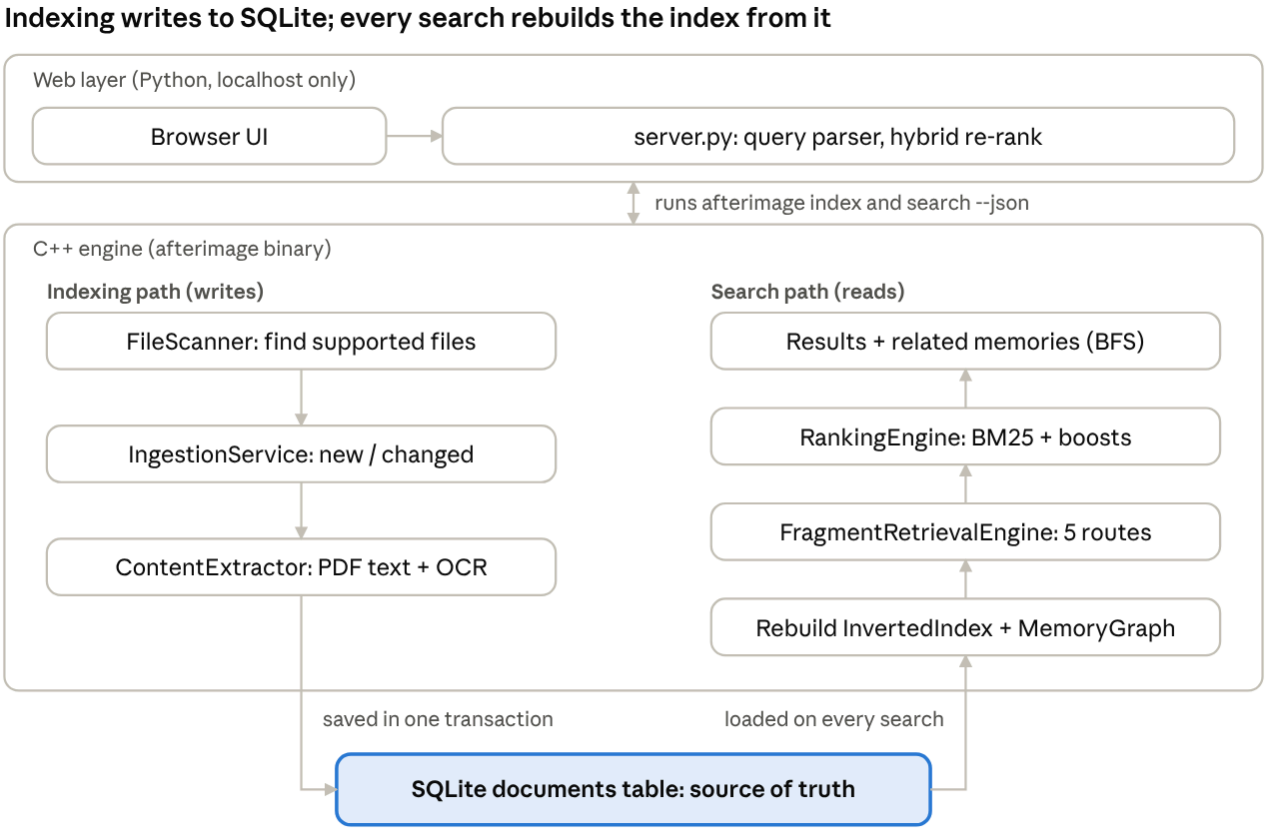
Type	Requirement
Optional	Python 3 for the web UI; Ollama with the nomic-embed-text model for semantic search

2.4 Technology stack and why each piece was chosen

Technology	Used for	Reason
C++20	Engine: scanning, extraction, index, retrieval, ranking, graph	Speed, control over memory layout, standard filesystem and string_view support
STL containers	std::map (ordered vocabulary), unordered_map, unordered_set, vector, queue	Ordered map gives prefix search through lower_bound; hash maps give O(1) lookups
SQLite3	Source of truth for documents	Single-file, transactional, no server
Poppler and Tesseract	PDF text and OCR	Mature open-source tools, launched as separate processes for safety
CMake and CTest	Build and tests	Portable build, 12 registered test groups
Python 3 (standard library only)	HTTP adapter, query parser, hybrid ranking	Fast to write natural-language rules; no dependencies to install
HTML, CSS, JavaScript	Browser interface	No framework; runs offline
Ollama (optional)	Local text embeddings	Semantic search without sending data to a cloud service
Swift (macOS only)	Launcher app and Spotlight bridge	Lets Afterimage appear in Spotlight

3. System architecture

AFTERIMAGE is a layered system. The C++ engine does the heavy work and is usable alone from the terminal. The Python adapter calls the engine as a subprocess, adds natural-language understanding and hybrid ranking, and serves the browser UI.



AFTERIMAGE architecture · web layer, C++ engine with two paths, SQLite

Read the left lane top to bottom to see how files become stored text, and the right lane bottom to top to see how a search turns stored text into ranked results plus related documents. SQLite is the only place where the two paths meet.

3.1 Two data paths

Indexing path (write). Folder, then scanner, then ingestion (new/changed/known files), then extraction (text, PDF, OCR), then SQLite. Only changed or new files are re-extracted.

Search path (read). SQLite loads all documents into memory, the inverted index and the concept graph are rebuilt in memory, then retrieval produces candidates, ranking orders them, and the graph adds related memories. SQLite stays the single source of truth; the index is derived data.

3.2 The eight engine modules

#	Module	Source file	Responsibility
1	Core document model	core/DocumentManager.cpp	Document record, stable IDs, in-memory library

#	Module	Source file	Responsibility
2	Ingestion	ingestion/FileScanner.cpp, IngestionService.cpp	Recursive scan, filtering, duplicate and change detection
3	Extraction	extraction/ContentExtractor.cpp, ExtractionService.cpp, TextNormaliser.cpp	Text, PDF and OCR extraction with safety limits
4	Persistent storage	storage/SQLiteDocumentStore.cpp	SQLite save and load with transactions
5	Text indexing	indexing/Tokeniser.cpp, InvertedIndex.cpp	Tokens, postings with positions, prefix, infix, fuzzy, phrase
6	Fragment retrieval	retrieval/FragmentRetrievalEngine.cpp	Combines the five retrieval routes into weighted candidates
7	Ranking	ranking/RankingEngine.cpp	BM25 plus title, path, phrase, coverage and recency boosts
8	Memory graph	graph/MemoryGraph.cpp	Document-concept graph and related-document search

Each module has a header in include/afterimage/<module>/ and one unit-test file, which is why the project can be tested module by module.

3.3 Project structure

```
AFTERIMAGE/
  CMakeLists.txt        build and 12 registered tests
  include/afterimage/   public headers, one folder per module
  src/                  core, ingestion, extraction, storage, indexing,
                        retrieval, ranking, graph, main.cpp (CLI)
  web/                  server.py (HTTP + query parser), app.js, index.html,
style.css
  macos/                AfterimageLauncher.swift, AfterimageSpotlight.swift
  tests/                unit/, integration/, fixtures/ (PDF, PNG),
test_50_queries.py
  data/                 sample_documents/, advanced_test_corpus/ (16 docs),
test_corpus_50/
```

3.4 Design principles used

- **Separation of concerns.** Each module has one job and talks to the next through plain structs (Document, SearchResult, RetrievalResult, RankedResult, RelatedDocument).
- **Derived data is disposable.** The index and graph are rebuilt from SQLite, so they can never disagree with the stored text.
- **Fail soft.** A damaged PDF or missing tool produces a status (Failed, DependencyMissing, TooLarge, Partial) and indexing continues with the other files.
- **Explainability.** Every result carries the route that found it and the score components that ranked it.
- **Safety by default.** External tools are launched without a shell, symlinks are not followed, hidden files are skipped, and the web server binds to localhost only.

4. Algorithms and core logic

This is the technical heart of the project. Each subsection gives the idea, the steps, an example and the cost.

4.1 Tokenisation (Tokeniser.cpp)

Idea: break text into lowercase words and remember each word's position, counted in words, not characters. Adjacent positions are what later prove that a phrase matches.

1. Scan the text one byte at a time.
2. A byte is part of a word if it is an ASCII letter or digit, or has a value of 128 or more (so UTF-8 words are not cut into pieces).
3. Lowercase ASCII letters; every other byte ends the current token.
4. Store each token with its position (0, 1, 2, ...).

Example: "Raft elects a Leader." gives raft(0), elects(1), a(2), leader(3). **Cost:** O(n) for n bytes.

4.2 Positional inverted index (InvertedIndex.cpp)

Idea: instead of reading every document for every query, store a dictionary from each word to the documents that contain it, and the positions where it occurs.

```
postingsByTerm_ : std::map<string, vector<Posting>>      (sorted by term)
  Posting       : { documentId, vector<position> }
termsByDocument_  : unordered_map<id, vector<term>>       (for fast removal)
lengthByDocument_ : unordered_map<id, tokenCount>          (needed by BM25)
```

Adding a document: (1) remove any old postings for the same ID so re-indexing never duplicates; (2) tokenise; (3) group positions by term in a map; (4) append one Posting per term; (5) record the document's term list and length, and update the running total length (used for the average document length).

Why std::map and not unordered_map for the vocabulary? A sorted map keeps terms in alphabetical order, which makes prefix search possible without scanning (next subsection). **Cost:** adding is O(n + u log V) for n tokens, u unique terms and vocabulary size V. Looking up one term is O(log V).

4.3 Prefix search (expandPrefix)

A query word such as "travers" should find "traversal" and "traversing". Because terms are sorted, every word starting with the prefix sits next to each other in the map.

1. Call lower_bound(prefix) to jump to the first term not smaller than the prefix.
2. Walk forward while the term still starts with the prefix; stop at the first term that does not, or at the expansion limit (20).

Cost: O(log V + k) for k matches. This is the main reason the vocabulary is an ordered map.

4.4 Infix search (expandContaining)

For fragments from the middle of a word ("ntersection" finds "intersections"), a sorted map cannot help, so the engine scans the vocabulary and uses string find. Matches where the fragment appears earlier in the word rank first, then alphabetical order. It only runs for fragments of 3 or more characters and returns at most 8 terms, which bounds the cost at O(V times L).

4.5 Typo-tolerant search with edit distance (expandFuzzy)

Idea: two words are "close" if few single-character edits (insert, delete, substitute) turn one into the other. This is the Levenshtein distance, computed by dynamic programming.

```
D(i,j)=\min\begin{cases}D(i-1,j)+1\\ D(i,j-1)+1\\ D(i-1,j-1)+[a_i\ne
b_j]\end{cases}\qquad D(i,0)=i,\; D(0,j)=j
```

The implementation (boundedEditDistance) is optimised in three ways:

- **Length filter:** if the lengths differ by more than the limit (1), the distance cannot be within the limit, so it returns at once.
- **Two rows only:** it keeps just the previous and current DP row, so memory is O(L) instead of O(L squared).

- **Early exit:** if the smallest value in a row already exceeds the limit, the word pair is rejected without finishing the table.

Example: "schedulng" to "scheduling" is one insertion, distance 1, so it matches. The CLI confirms this: `afterimage search "schedulng"` reports "fuzzy fragment 'schedulng' matched 'scheduling'". Fuzzy search only runs for words of 4 or more letters.

4.6 Phrase search with positions (phraseOccurs)

A phrase such as "raft leader" must appear as consecutive words, not just both words somewhere in the document.

1. Take the posting list of the first word; only documents in it can match.
2. For each document, fetch the position lists of all phrase words. If any word is missing, the document fails.
3. For every start position p of the first word, check that word k occurs at position p + k, using binary search on the sorted position list.
4. The first start position where all offsets succeed proves the phrase.

Cost: O(P1 times k times log P) where P1 is the number of occurrences of the first word. Storing positions in the index is exactly what makes this possible.

4.7 Fragment retrieval engine (FragmentRetrievalEngine.cpp)

This module does not rank; it gathers candidates and records why each one was found. For every distinct query word it tries several routes, each with a reliability weight, and sums the evidence per document.

Route	What it matches	Weight	Condition
Exact	the word itself	1.00	always
Prefix	words starting with it	0.85	up to 20 expansions
Infix	words containing it	0.65	fragment 3+ characters, up to 8 expansions
Fuzzy	words one edit away	0.45	only if no exact, prefix or infix match was found; 4+ characters
Phrase	all words consecutive	1.25	query has 2+ words

Two design decisions matter for the viva. First, **fuzzy is a fallback**, used only when stronger routes found nothing, so it cannot add noise to a good query. Second, each (route, query word, matched term) triple is stored once, so the same evidence is never counted twice. The result keeps a list of RetrievalMatch records, which becomes the "Matched by" line shown to the user.

4.8 Why retrieval and ranking are two stages

The index's own search function scores with a simple weighted tf-idf. That is only used to decide which documents are candidates and how strong each route's evidence is. The final order comes from a separate ranking stage that recomputes a proper BM25 score from the index statistics. Splitting the stages lets the project change the ranking formula without touching the index.

4.9 Ranking with BM25 and explainable boosts (RankingEngine.cpp)

BM25 is the standard relevance formula in search engines. It rewards words that are frequent in a document but rare in the library, and it stops long documents from winning just because they are long.

```
\mathrm{BM25}(d,q)=\sum_{t}\rho_t\cdot \ln\!\Big(1+\frac{N-n_t+0.5}
{n_t+0.5}\Big)\cdot\frac{f_{t,d}\,(k_1+1)}{f_{t,d}+k_1\big(1-b+b\,\frac{|d|}
{\mathrm{avgdl}}\big)}
```

Here N is the number of documents, n_t the number containing term t, f is the term's count in document d, |d| the document length, avgdl the average length, k1 = 1.2 and b = 0.75. The extra factor rho is the **route reliability** of the match: 1.0 for exact, 0.88 for prefix, 0.72 for infix and 0.58 for fuzzy, so a typo match contributes less than an exact one. For each matched vocabulary term only the strongest route is counted.

The final score adds small, named boosts:

```
\mathrm{score}=\mathrm{BM25}+1.25\,H_{title}+0.25\,H_{path}+0.75\,
[\text{phrase}]+0.5\,C+0.2\cdot 2^{-\,\mathrm{age}/90}
```

Component	Meaning
H_title	number of matched terms found in the document title (a term in the title is not counted again as a path hit)
H_path	matched terms found only in the file path
phrase	1 if the whole query occurs as a consecutive phrase

Component	Meaning
C	query coverage: matched query words divided by total query words
age term	recency: halves every 90 days since the file was modified; can be switched off with --no-recency

Because each component is stored in a RankingBreakdown, the CLI and the web UI can print "BM25=..., title+=..., recency+=..." for any result. That is the explanation feature.

4.10 Memory relationship graph (MemoryGraph.cpp)

Idea: two documents are probably related if they share rare, informative words. The graph is bipartite: document nodes on one side, concept nodes on the other, an edge when a document uses a concept.

Building the graph

1. For each document, collect candidate concepts from the title and body. Words shorter than 4 letters and about 200 stop-words (for example "system", "data", "process") are dropped.
2. Keep a body word only if it appears at least twice or appears in the title, which avoids accidental one-off links.
3. Weight = frequency + 2 times title frequency. Keep the top 120 concepts per document.
4. Count how many documents use each concept. Remove concepts that appear in more than 25 percent of the library (at least 2 documents), because such "hub" words connect everything and mean nothing.

Finding related documents. The top 3 search results are the seeds. A breadth-first search (BFS) with a queue walks seed, then concept, then neighbour document; each such step counts as 2 edges. With the default limit of 2 hops, results are documents that share at least one concept with a seed.

```
\mathrm{idf}(c)=\ln\!\Big(1+\frac{N}{df(c)}\Big)\qquad \mathrm{score}
(d)=\sum_{c\in \mathrm{shared}(d)}\frac{\mathrm{idf}(c)}{\mathrm{hops}(c)/2}
```

A rare shared concept therefore counts for more than a common one, and a longer path counts less. The five strongest connecting concepts are shown as the explanation (for example "Connected through: distributed, failure"). The graph is honest about its limits: it is a lexical heuristic, not a knowledge graph, and a shared word is evidence of a relationship, not proof.

4.11 Natural-language query understanding (web/server.py, MemoryQueryParser)

This Python class turns a human sentence into a structured query. For "A java assignmnet that i opened last week" it produces core terms [java, assignment], a corrected typo, and a time window.

1. **File-type detection.** A regular expression finds extensions or media words (pdf, png, jpg, md, txt, images, scan) and sets a file-type constraint.
2. **Time detection.** Expressions are mapped to epoch-second windows (table below) and then removed from the text.
3. **Filler removal.** About 20 patterns strip conversational words ("I remember seeing", "find that", "hey, can you") so only the remembered subject is left.
4. **Typo correction.** A dictionary of common misspellings maps "schedulng" to "scheduling" and "assignmnet" to "assignment".
5. **Stop-word filter** leaves the core terms.
6. **Concept expansion.** Synonyms (resume to cv), a small concept taxonomy ("authentication" to identity, password, mfa) and rule-based phrases ("duplicate data" to unique, primary key, normalization) add related words. Expansions only add candidates; they never replace the user's own words.
7. **Explanation.** The parser also builds the readable sentence shown above the results ("Focusing on concepts ... Corrected spelling ... Time filter ...").

Expression	Window used	Time weight
2026-03-14 (exact date)	that calendar day	4.0
yesterday, today	the last 2 days / 1 day	4.0
last week	the previous Monday to Sunday	3.5
last month	the previous calendar month	3.0
recently, a few days ago	the last 5 days	3.0
in 2025	that calendar year	2.5
past week, past month	rolling 7 / 30 days	3.5 / 3.0

4.12 Hybrid re-ranking (perform_intelligent_search)

The web layer runs the C++ engine with --json, then merges three candidate sources and re-scores them in Python:

- the engine's lexical results;

- a direct SQLite check of title, path, tags and notes (title 4.5, tags 4.0, notes 3.5, path 3.0 points);
- local-AI semantic matches, when an embedding index exists.

If the literal words find nothing, the engine is called again with up to 6 expansion words. Each candidate's score is then adjusted:

Signal	Effect on score
Engine score (BM25 etc.)	base value
Semantic cosine similarity	plus 28 times similarity
Fraction of core terms present as whole words	plus 35 times the fraction
All terms present (2+ terms)	plus 25
Exact multi-word phrase in text	plus 40
Date inside the requested window	plus 18 times the window weight; outside the window the file is dropped
File type does not match request	file is dropped
Related-concept hits in text	plus 2.2 per hit, capped at 12
Path inside node_modules, _deps, .git or cmake-build	multiplied by 0.12 (dependency noise)

The best result is shown as about 98 percent and others are scaled against it, clamped between 45 and 99 percent. This is a relative confidence for display, not a probability.

4.13 Page and passage extraction (extract_best_passage)

To tell the user where to look, each candidate's text is split into pages: on form-feed characters when the text came from a PDF, otherwise into virtual pages of 2,500 characters. Each page scores 3 points per occurrence of a query term plus 1 per related concept; the best page wins. A 280-character window starting 80 characters before the first hit is cut, HTML-escaped, and the matches are wrapped in highlight tags (primary words and concept words get different styles).

4.14 Semantic search with local embeddings (SemanticIndexer)

Keyword methods cannot match "keeps copies of frequently used data" with a document that says "cache". Embeddings can, because texts with similar meaning have similar vectors.

1. For each document, build the text "Title … Content …" (first 12,000 characters) and hash it with SHA-256.
2. If the hash differs from the stored one, ask the local Ollama model (nomic-embed-text) for a vector, in batches of 8, and store it as a float32 BLOB in the semantic_vectors table. Unchanged documents are skipped.
3. At search time, embed the query and compute the cosine similarity with every stored vector; keep the top 60.

```
\cos(\mathbf{q},\mathbf{d})=\frac{\mathbf{q}\cdot\mathbf{d}}
{\lVert\mathbf{q}\rVert\,\lVert\mathbf{d}\rVert}
```

If Ollama is not running, semantic search silently returns nothing and the system continues with lexical search only.

4.15 Incremental indexing and change detection

On every index run, the scanner records each file's size and modification time. A path is normalised with weakly_canonical so the same file reached by two routes is one document. If a known path has a different size or time, the stored text is cleared and the file is queued for re-extraction; if both are unchanged it is skipped, so slow OCR is never repeated needlessly. The flag --force re-extracts everything.

4.16 Complexity summary

Operation	Time cost	Note
Tokenise a document	O(n)	n = bytes
Add a document to the index	O(n + u log V)	u = unique terms
Exact term lookup	O(log V)	ordered map
Prefix expansion	O(log V + k)	k = matches returned
Infix expansion	O(V times L)	bounded to 8 results, 3+ characters
Fuzzy expansion	up to O(V times L squared), pruned	length filter and early row exit
Phrase check	O(P1 times k times log P)	binary search on positions
BM25 for one candidate	O(m times df)	m matched terms; term frequency needs a posting scan

Operation	Time cost	Note
Graph build	O(N times T)	T = concepts per document (max 120)
Related search	O(seeds times concepts times df)	bounded by 2 hops

5. Implementation details

5.1 Core document model (Document.h, DocumentManager.cpp)

A Document holds id, title, filePath, fileType, extractedText, sizeBytes and modifiedAtEpochSeconds. DocumentId is a named 64-bit unsigned type, so an ID is never confused with a count.

DocumentManager stores documents in a vector and keeps an unordered_map from ID to vector position for O(1) lookup. New IDs come from a counter. When the library is reloaded from SQLite, restoreDocument keeps the saved ID and moves the counter past it, so IDs are stable across restarts and are never reused. addDocument inserts the lookup entry first and rolls it back if the vector insertion throws, so the two containers cannot disagree (exception safety). updateFileMetadata clears the old text when size or time changed.

5.2 Ingestion (FileScanner.cpp, IngestionService.cpp)

- Uses std::filesystem::recursive_directory_iterator with skip_permission_denied.
- Symbolic links are counted and skipped, so a link cannot escape the chosen folder or create a cycle.
- Hidden entries (names starting with a dot, such as .DS_Store) are skipped, and hidden folders are not entered (disable_recursion_pending).
- Only supported extensions are kept: txt, md, markdown, pdf, png, jpg, jpeg, tif, tiff, bmp, webp, gif.
- Every problem (unreadable entry, bad size) becomes a ScanIssue in the report instead of stopping the scan.
- File time is converted from the filesystem clock to Unix time with a clock-offset trick, because the two clocks do not share an epoch in C++20.
- The IngestionReport returns counts: new, already known, changed, unsupported, hidden and symlinks skipped.

5.3 Extraction (ContentExtractor.cpp)

Input	Method	Fallback or limit
TXT, MD	Read bytes, strip UTF-8 byte-order mark, convert line endings to \n	50 MiB input limit, 12 MiB text limit (status Partial)
PDF with text	pdftotext with UTF-8 and layout mode	120 s timeout
Scanned PDF	If no text came back: pdftoppm renders pages to PNG (max 1800 px), Tesseract reads each page	first 20 pages, status Partial if more exist
Images	Tesseract with language and page-segmentation mode 3	120 s timeout

The external tools are started by a small runProcess function built on posix_spawn with a pipe for output. It never builds a shell command, so a filename containing spaces or shell characters cannot inject commands. It reads the pipe without blocking, caps the output size, and kills the child with SIGKILL on timeout. Every outcome is an ExtractionStatus: Success, Partial, Empty, Unsupported, DependencyMissing, Failed or TooLarge. ExtractionService wraps each file in try/catch so one bad file never aborts the whole run. TextNormaliser removes the byte-order mark, unifies line endings and trims trailing blanks.

5.4 Persistent storage (SQLiteDocumentStore.cpp)

```
CREATE TABLE documents (
  id INTEGER PRIMARY KEY,
  title TEXT NOT NULL,
  file_path TEXT NOT NULL UNIQUE,
  file_type TEXT NOT NULL,
  extracted_text TEXT NOT NULL DEFAULT '',
  size_bytes INTEGER,
  modified_at_epoch_seconds INTEGER
);
CREATE INDEX idx_documents_file_type ON documents(file_type);
```

The web layer adds columns tags, user_notes, extraction_method, page_count and indexed_at_epoch_seconds, and a second table semantic_vectors (document_id, content_hash, dimensions, vector BLOB, indexed_at).

Implementation points worth mentioning:

- **RAII wrapper** class Statement prepares a statement in the constructor and finalises it in the destructor, so a thrown error cannot leak a prepared statement.
- **Bound parameters** (sqlite3_bind_*) are used everywhere; text is never pasted into SQL.
- **Upsert** with INSERT … ON CONFLICT(id) DO UPDATE saves new and changed documents with one statement.
- **Transactions.** saveAll runs BEGIN IMMEDIATE, saves every document, then COMMIT; on any exception it ROLLBACKs, so the database is never half-written.
- A busy timeout of 5 seconds and the full-mutex open flag let the CLI and the web server touch the same file safely.
- Optional values (size, time) are stored as NULL, not as fake zeros.

5.5 Command-line interface (main.cpp)

```
afterimage index <folder> [--force] [--no-ocr] [--ocr-language hin+eng]
afterimage search "fragment" [--max-results N] [--phrase] [--all-terms]
                  [--exact-only] [--no-prefix] [--no-infix] [--no-fuzzy] [--
no-recency] [--json]
afterimage            interactive prompt with :stats, :help, :quit
```

The index command runs: restore library from SQLite, ingest folder, extract only new or changed files, save in one transaction, rebuild the index and print a report. The search command runs: load, build index and graph, retrieve, rank, take the top 3 results as graph seeds, print results and up to 5 related memories. With --json the same results are printed as JSON for the web layer, including the score breakdown and a preview.

The database path is resolved in this order: --db, the AFTERIMAGE_DB environment variable, ~/.afterimage/library.db (with fallback to an older data/afterimage.db if that exists).

5.6 Web adapter (web/server.py)

A standard-library HTTP server bound to 127.0.0.1:8765. It never talks to the index directly; it calls the C++ binary with argument lists, which keeps the engine as the single implementation of search.

Method	Endpoint	Purpose
POST	/api/search	Natural-language or exact-word search with filters

Method	Endpoint	Purpose
POST	/api/index, /api/index/cancel	Start or cancel background indexing of a folder
GET	/api/index/progress	Progress, current step and live log lines
GET	/api/status, /api/stats, /api/table, /api/document	Library status, statistics, listing, one document with pages
POST	/api/metadata	Edit title, tags and personal notes
POST	/api/semantic/index, GET /api/semantic/status	Build and monitor the local-AI vector index
POST	/api/spotlight/sync, /api/open	macOS Spotlight bridge, open a file
POST	/api/sql	Developer console; only SELECT, PRAGMA and EXPLAIN are allowed

BackgroundIndexer runs the engine in a worker thread, streams its output into a log buffer guarded by a lock, supports cancellation by terminating the process, and afterwards labels each document with its extraction method and an estimated page count.

5.7 Browser interface (index.html, app.js, style.css)

The UI has three views. **Memory Search** has the search box, a "how I understood your query" bar, date and format filters, ranked cards with highlighted passage, page number, score explanation and related memories. **Library** starts folder indexing with a live progress bar and log, and lists all documents with filtering and paging. **Developer Console** shows statistics (documents, database size, terms, postings, concepts, edges), buttons for the local-AI index and Spotlight sync, and a read-only SQL workbench with preset queries. Clicking a result opens an in-browser reader with page-by-page text and an extraction inspector; titles, tags and notes can be edited there.

5.8 Security and robustness measures

Risk	Measure in the code
Shell injection through file names	Tools launched with posix_spawn and argument arrays, never a shell string
Hanging or huge external tool	120 s timeout, output cap, SIGKILL

Risk	Measure in the code
Memory exhaustion	50 MiB input limit, 12 MiB text limit, 20-page OCR cap
Symlink loops or escaping the folder	Symlinks never followed
Corrupt database state	Transactions with rollback; bound parameters
Remote access to the web UI	Bound to 127.0.0.1 only; JSON request size limited
Destructive SQL from the console	Only SELECT, PRAGMA and EXPLAIN accepted
One bad file stopping a run	Per-file try/catch and explicit statuses

6. Testing and results

6.1 Automated test suite

The project registers 12 CTest groups and all 12 pass. I rebuilt the project from the submitted source on Linux and ran them; the whole suite took about 4 seconds. The test files contain about 290 assertions in total.

#	Test group	What it proves
1	DocumentManagerTests	IDs are unique and stable; metadata refresh clears stale text
2	IngestionTests	Recursive scan, extension filter, hidden files and symlinks skipped, duplicates and changes detected
3	StorageTests	Save, reload, upsert and transaction behaviour in SQLite
4	ExtractionTests	Plain text, PDF text layer, scanned-PDF OCR, image OCR, size limits, missing tools
5	IndexingTests	Tokenisation, positions, prefix and phrase search, removal and re-indexing
6	RetrievalTests	Exact, prefix, infix, fuzzy and phrase routes and their evidence
7	RankingTests	BM25 ordering, title, path, phrase, coverage and recency boosts

#	Test group	What it proves
8	GraphTests	Concept extraction, common-word filtering, shared-concept ranking, seed exclusion, result limits
9	PipelineIntegrationTests	End to end on the 16-document technical corpus: ingest, extract, save, reload, index, retrieve, rank, graph
10	CommandLineHelp	The binary starts and prints help
11	CommandLineWorkflow	index, then search, then re-index is idempotent
12	CommandLineMediaWorkflow	PDF and image fixtures are extracted and searchable

Tests use fixed timestamps for ranking so that results are deterministic, and checked-in PDF and PNG fixtures for OCR.

6.2 Sample outputs

Query typed	What the system did	Observed result
schedulng	No exact word exists, so the fuzzy route ran	Top result "fragment_retrieval_demo" via "fuzzy fragment 'schedulng' matched 'scheduling'"
ntersection	Middle-of-word fragment, infix route	Matches "intersections" in distributed_consensus_deep_dive
raft leader heartbeat	Exact plus prefix (heartbeats, leaders, leadership) plus phrase evidence	raft_consensus scores 21.76; the next document scores 6.81
scaled dot product attention	Multi-word query across the technical corpus	transformer_attention_mechanisms appears among the top results

For the Raft query the engine also listed related memories such as distributed_transactions, connected through the shared concepts "distributed" and "failure".

6.3 The 50-query memory benchmark

tests/test_50_queries.py runs 50 human-style queries in five groups: vague recollections, different wording, messy conversational queries, PDF and OCR queries, and relationship queries. Its pass condition is that at least one result is returned.

Library used	Documents	Searchable terms	Queries with results	Average time per query
Bundled sample and test corpora only	25	1,695	26 of 50	about 4 ms
The author's own library (data/afterimage.db)	2,185	54,245	30 of 50	about 60 ms

How to read these numbers honestly:

- The pass condition only checks that something was returned, not that the right document is first. A proper evaluation would label the correct document for each query and report precision at 1 or 3 and mean reciprocal rank.
- On the bundled corpus 24 queries return nothing (7, 8, 10, 16, 18, 19, 22 to 26, 29 to 35, 37 to 40, 48 and 49). Most of these ask for scanned PDFs, diagrams, statistics notes or Java stream and RAM/ROM material that only exist in the author's personal library, which is not in the submitted bundle. Two are genuine misses: query 7 (classes and objects) and query 48 (neural networks) ask about topics that the bundled java_oop_assignment and neural_net_architectures files do contain, so they show where the rule-based query parser needs improvement.
- The author's own library also contains thousands of unrelated files that were indexed from other folders (documentation inside node_modules, build directories). This adds noise: for example "scaled dot product attention" ranks an unrelated Next.js documentation page first and the correct transformer document second. The "dependency noise" penalty (multiply by 0.12) reduces this but does not remove it.

For a clean demonstration, index only data/advanced_test_corpus, data/sample_documents and data/test_corpus_50 into a fresh database (see Appendix A).

7. Limitations

- **Lexical, not semantic, by default.** Meaning-based matching needs the optional Ollama model.

- **Index is rebuilt in memory on every search session.** This is simple and always consistent, but the cost grows with library size. A persistent on-disk index is the next optimisation.
- **OCR quality limits text quality.** Handwriting, diagrams, low-resolution scans and complex tables are weak; scanned PDFs stop at 20 pages by default.
- **Change detection uses size and modification time.** A file edited without changing either value is missed until --force is used.
- **Rule-based query parser.** The typo dictionary, concept taxonomy and phrase rules are hand written and partly tuned to the 50 test queries, so unseen topics get less help. The C++ fuzzy route still corrects general one-letter typos.
- **Graph is a heuristic.** A shared word is evidence, not proof, of a relationship.
- **Platform.** The process runner uses POSIX calls, and Spotlight and the launcher app are macOS only.
- **No stemming.** "schedule" and "scheduling" connect only through prefix or fuzzy matching.

8. Future scope

1. Persist the inverted index and graph on disk so search starts instantly on large libraries.
2. Add stemming or lemmatisation and BM25F (separate weights for title and body fields).
3. Combine lexical and embedding rankings with reciprocal rank fusion and build a labelled test set with precision@k and MRR.
4. Extract people, places and events to build a real knowledge graph, and read dates from document content.
5. Add handwriting-capable OCR and layout-aware table extraction.
6. Windows support, file-system watchers for automatic re-indexing, and optional encryption of the database.

9. Conclusion

AFTERIMAGE shows how classic data-structure and information-retrieval ideas fit together into a practical tool: a positional inverted index on an ordered map, edit-distance search with pruning, BM25 ranking with transparent boosts, and a concept graph searched with breadth-first search. Around that core, careful engineering (transactions, RAII, no-shell process launching, resource limits, 12 passing test groups) makes it robust,

and the browser layer makes it usable. The project meets its objectives of retrieving documents from imperfect memory, explaining every result, and keeping all data private on the user's machine.

10. References

1. S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond", Foundations and Trends in Information Retrieval, 2009.
2. C. Manning, P. Raghavan and H. Schutze, Introduction to Information Retrieval, Cambridge University Press, 2008 (inverted index, tf-idf, spelling correction, phrase queries).
3. V. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals", Soviet Physics Doklady, 1966.
4. T. Cormen, C. Leiserson, R. Rivest and C. Stein, Introduction to Algorithms (BFS, dynamic programming, balanced search structures).
5. R. Smith, "An Overview of the Tesseract OCR Engine", ICDAR, 2007.
6. SQLite documentation (prepared statements, transactions, upsert); Poppler utilities documentation; ISO C++20 standard (std::filesystem, string_view).
7. Ollama and the nomic-embed-text embedding model documentation (local embeddings).

Appendix A. How to build, run and demonstrate

```
brew install cmake sqlite poppler tesseract        # macOS; use apt on Linux
cmake -S . -B build -DCMAKE_BUILD_TYPE=Debug
cmake --build build -j2
ctest --test-dir build --output-on-failure         # expect 12 of 12 passed

export AFTERIMAGE_DB=/tmp/demo.db                  # fresh, clean database
./build/afterimage index data                      # 25 documents, 1,695
terms
./build/afterimage search "schedulng"              # fuzzy
./build/afterimage search "ntersection"            # infix
./build/afterimage search "raft leader heartbeat"  # exact + prefix + phrase
+ graph
python3 web/server.py                              # browser UI at
http://127.0.0.1:8765
```

Suggested demo order: show the CLI fuzzy and infix searches, show the score explanation, show related memories, then open the web UI and try "A java assignmnet that i opened last week" to show typo correction, time filtering and the interpretation bar.

Appendix B. Likely viva questions and short answers

Question	Short answer
Why an inverted index?	It maps each word to the documents containing it, so a query reads only a few posting lists instead of scanning every document.
Why store positions?	They allow exact phrase search: word k must be at position p + k.
Why std::map for the vocabulary?	Sorted order makes prefix search O(log V + k) with lower_bound.
How does typo tolerance work?	Levenshtein distance with a length filter, two-row DP and early exit, allowing one edit, and only when stronger routes found nothing.
Why BM25 instead of plain tf-idf?	It saturates term frequency and normalises by document length, so long or repetitive documents do not dominate.
What do k1 and b do?	k1 (1.2) controls how fast repeated words stop adding score; b (0.75) controls how strongly long documents are penalised.
How is a typo match scored lower?	Each route has a reliability factor (exact 1.0, prefix 0.88, infix 0.72, fuzzy 0.58) multiplied into the BM25 term score.
How does the graph avoid meaningless links?	Stop-words, short words, one-off words and concepts used in more than 25 percent of documents are removed; rare shared concepts score higher through idf.
Why is SQLite the source of truth and the index in memory?	The index is derived data; rebuilding it guarantees it matches the stored text. The cost is startup time, solved later by an on-disk index.
How is OCR integrated safely?	Poppler and Tesseract run as separate processes through posix_spawn with argument arrays, time limits, output caps and explicit statuses.

Question	Short answer
How do you handle "last week"?	The Python parser converts it to an epoch window (previous Monday to Sunday), drops files outside it, and adds a time bonus to files inside it.
What is the semantic search?	Local embeddings from Ollama are stored as vectors; the query's vector is compared by cosine similarity and added to the lexical score.
What are the weaknesses?	Lexical by default, rule-based query parser, OCR limits, index rebuilt per session, benchmark only checks for non-empty results.